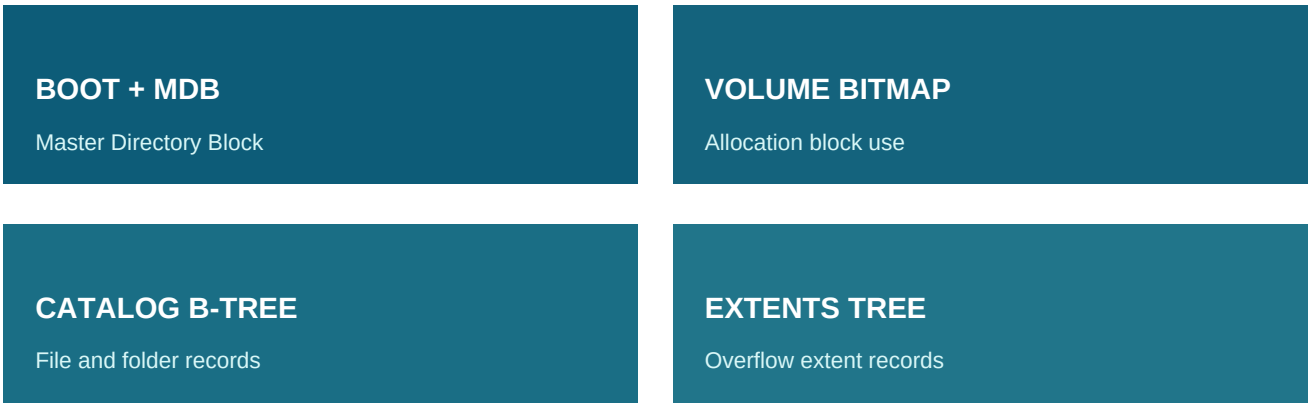
HFS - original Macintosh hierarchical filesystem

OFFICIAL FORMAT / CHRONOLOGY / ENGINEERING

Apple’s 1985 Hierarchical File System replaced the original flat Macintosh File System with a B-tree catalog and a true folder hierarchy.

This guide documents the original format’s technical architecture, history and independent source references. InfiltratorOS modules and install packages are implementation artefacts, not the authority for the historic on-disk definition.

CONCEPTUAL MEDIA MAP



CONTENTS

1. Historical chronology
2. Original on-disk architecture
3. Allocation and directories
4. Integrity and recovery
5. Design rationale and limitations
6. InfiltratorOS integration boundary

HFS - original Macintosh hierarchical filesystem

HISTORICAL CHRONOLOGY

Apple introduced the Hierarchical File System in 1985 alongside the Macintosh HD20 hard disk and Finder 5.0, replacing the flat Macintosh File System (MFS). Contemporary 1985 technical writing explains the need: linear search in a single directory was increasingly inefficient on larger disks with hundreds of files. A hierarchical B-tree catalog enabled practical nested folders and more scalable lookup.

HFS became the classic Mac OS volume format. Apple later introduced HFS Plus with Mac OS 8.1 in 1998 to solve allocation, naming and scale restrictions. HFS is now historically important for classic Macintosh media, archival disk images and compatibility, but the original format does not inherit the extended Unicode and allocation-block fields of HFS Plus.

HFS - original Macintosh hierarchical filesystem

ORIGINAL ON-DISK ARCHITECTURE

The HFS volume begins with boot blocks and a Master Directory Block (MDB) containing volume geometry and pointers. Critical structures include the volume bitmap, catalog B-tree, extents overflow B-tree and file-fork data areas. B-tree nodes and allocation units are format-specific and must not be confused with HFS Plus's revised structures.

HFS distinguishes a data fork and resource fork for each file. Finder metadata and creator/type codes support the Macintosh desktop model. The catalog associates parent directory identifiers and names with files and folders. CNIDs provide stable object identity through renames in ways that path-only formats do not.

HFS - original Macintosh hierarchical filesystem

ALLOCATION AND DIRECTORIES

The volume bitmap records free and allocated blocks. Fork extents are stored partly in catalog records with overflow extents maintained in a separate B-tree. Choosing large allocation blocks on growing disks causes significant slack for small files, a major reason for later HFS Plus development.

The catalog B-tree stores file and directory records ordered by parent identifier and name. Original HFS names use a Mac-encoding-oriented, 31-byte form, not the Unicode names later introduced by HFS Plus. Case-insensitive comparison follows classic Macintosh rules and must not be replaced with modern Unicode folding without explicit format logic.

HFS - original Macintosh hierarchical filesystem

INTEGRITY AND RECOVERY

Original HFS is not a modern copy-on-write filesystem and does not inherently include the later optional HFS Plus metadata journal. An interrupted update can desynchronise the catalog, extents overflow tree, bitmap and directory identity records. Disk repair tools must reconstruct consistency between forks, tree nodes and allocation state.

The MDB and alternate critical metadata provide recovery clues but do not guarantee that all file contents are current. A damaged catalog can obscure the file hierarchy even if data extents remain on the disk. Rebuilding tree indexes from guessed names or object identities can destroy surviving Macintosh metadata.

HFS - original Macintosh hierarchical filesystem

DESIGN RATIONALE AND LIMITATIONS

HFS’s B-tree catalog was a major practical improvement over MFS’s flat listing, enabling larger volumes and human-friendly folders. Dual forks, Finder metadata and CNIDs reflected Macintosh software conventions rather than the assumptions of DOS or Unix.

Restrictions on names and allocation-block counts became limiting as drives grew. HFS Plus reduced average slack and added improved Unicode support. HFS remains a distinct physical format with different tree, fork and block-size rules, even if an operating system exposes similar file APIs for both.

HFS - original Macintosh hierarchical filesystem

INFILTRATOROS INTEGRATION BOUNDARY

HFS.so must preserve data and resource forks, Finder metadata, catalog CNIDs and extents overflow semantics. A basic read-only file listing does not demonstrate full Macintosh interoperability. Conversions to ordinary POSIX files require decisions about resource-fork representation.

Apple’s historical technical publications are the authority for this format. HFSPlus.so is a separate adapter for the successor format, and the old HFS on-media records must not be interpreted using HFS Plus struct layouts.

HFS - original Macintosh hierarchical filesystem

PRIMARY SOURCE REFERENCES

These primary technical documents and near-contemporary references provide provenance for the design history and on-disk structures. For exact field sizes or a particular format revision, consult the original specifications and source code, not the conceptual cover illustration.

1. Contemporary 1985 HFS introduction (MacTech)

https://preserve.mactech.com/articles/mactech/Vol.02/02.01/HFS/index.html

2. Apple Technical Note TN1150, HFS and HFS Plus comparison

https://developer.apple.com/library/archive/technotes/tn/tn1150.html

Our independent .so, optional .ko, build procedures and test evidence describe a separate implementation. The source-level history in this PDF cannot itself qualify production writes, boot, recovery or cross-platform compatibility.

HFS / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Identify the HFS volume through the Master Directory Block and verify allocation block size, volume bounds and the catalog and extents overflow file locations.

2. Read the allocation bitmap before concluding which allocation blocks are free. Physical sectors and HFS allocation blocks use different numbering and sizes.

3. Reconstruct the catalog B-tree with its header, index and leaf nodes; catalog records contain files, folders and threading information.

4. Interpret catalog node IDs as stable object identities. File and folder names are indexed under parent directory identities rather than stored in a FAT-style entry chain.

5. Decode the data fork and resource fork separately. Finder metadata, type and creator codes belong to the Macintosh format’s original application model.

6. Use the extents overflow B-tree when the initial extent descriptors cannot represent all of a fork’s physical allocation.

7. Observe the classic HFS filename and script-manager encoding constraints; the stored name model is not identical to HFS Plus Unicode rules.

8. Check the alternate Master Directory Block where available, while recognising that an alternate copy is not a transaction journal.

HFS / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Validate B-tree node size, record offsets, sibling links and key ordering before trusting catalog or extents records.

2. Check that catalog records and thread records agree on parent and file identity; a valid node checksum is not a substitute for namespace consistency.

3. Map each fork’s declared logical length to its extent records and ensure the physical allocation stays within volume boundaries.

4. Compare bitmap allocation with fork extents and B-tree metadata. Duplicate ownership or orphaned allocations must be reported distinctly.

5. Treat original Macintosh time encoding and the Mac epoch separately from Unix time; incorrect epoch translation can produce plausible but wrong dates.

6. Do not apply HFS Plus volume-header or Unicode key rules to classic HFS. The two formats have related goals but different on-disk structures.

7. Preserve resource fork and Finder information during conversion; treating only the data fork as a complete Mac file causes information loss.

8. Metadata recovery must distinguish the primary and alternate MDB, catalog structure and extents overflow rather than assuming any single copy is authoritative.